THREE SIGMA

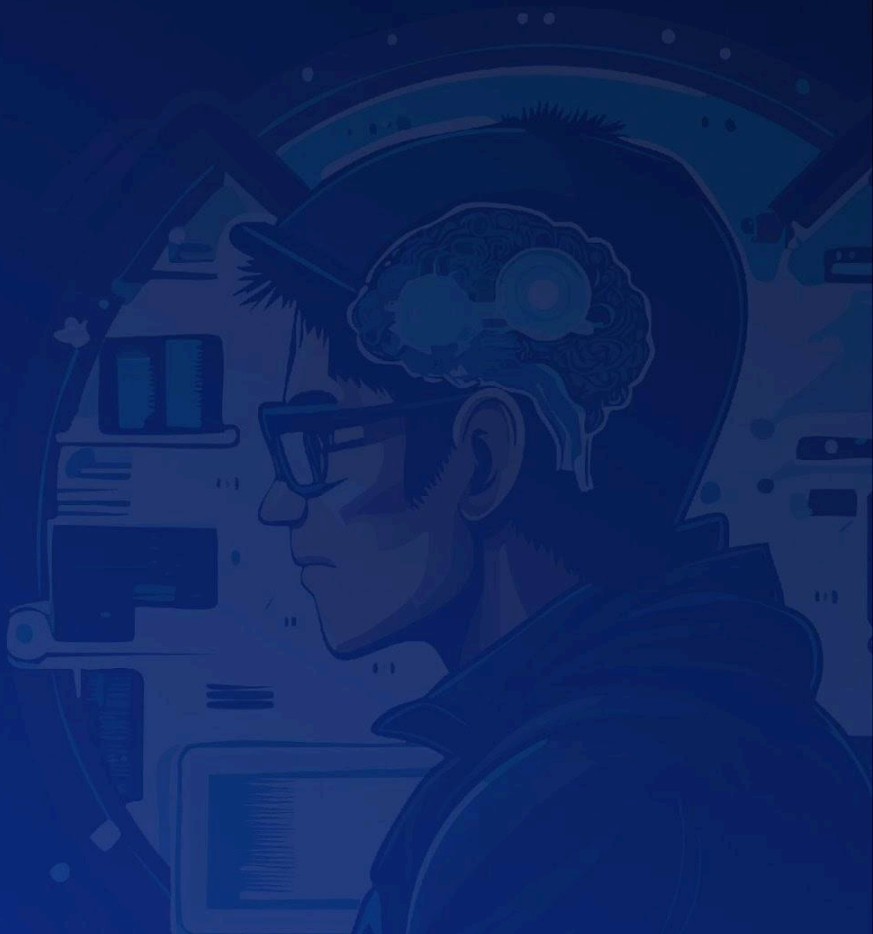
Felix Morpho-BLP adapter

Security Review

Disclaimer

Security Review

Felix Morpho-BLP adapter



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

Felix Morpho-BLP adapter

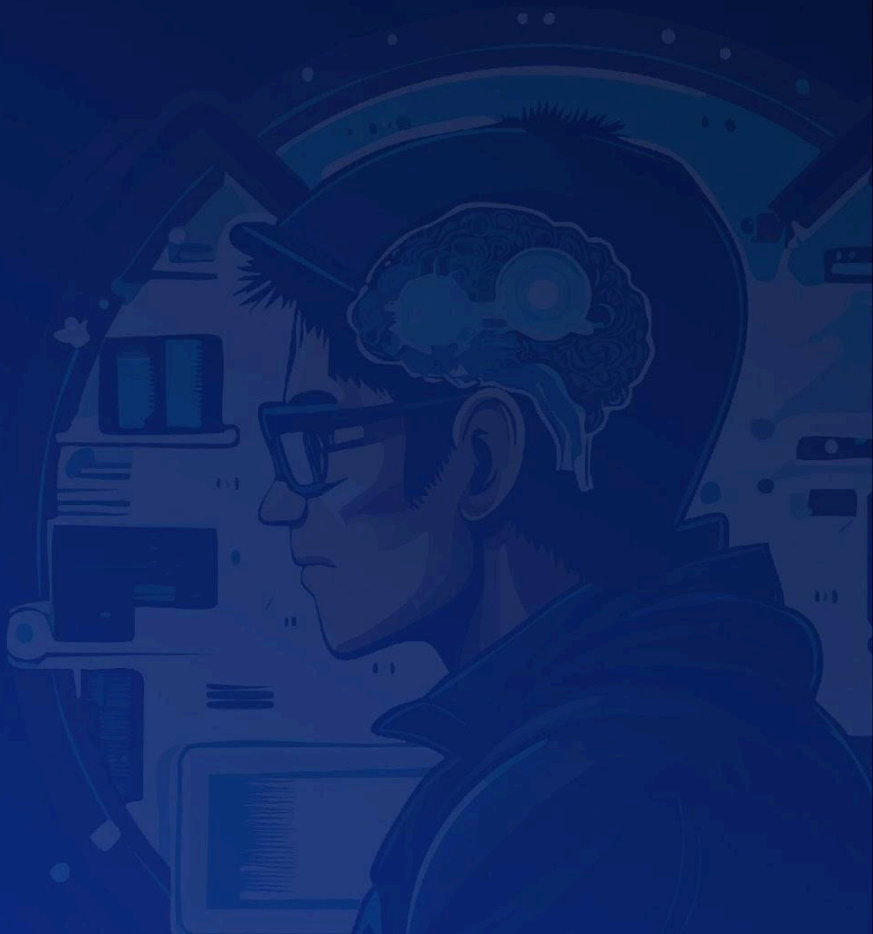
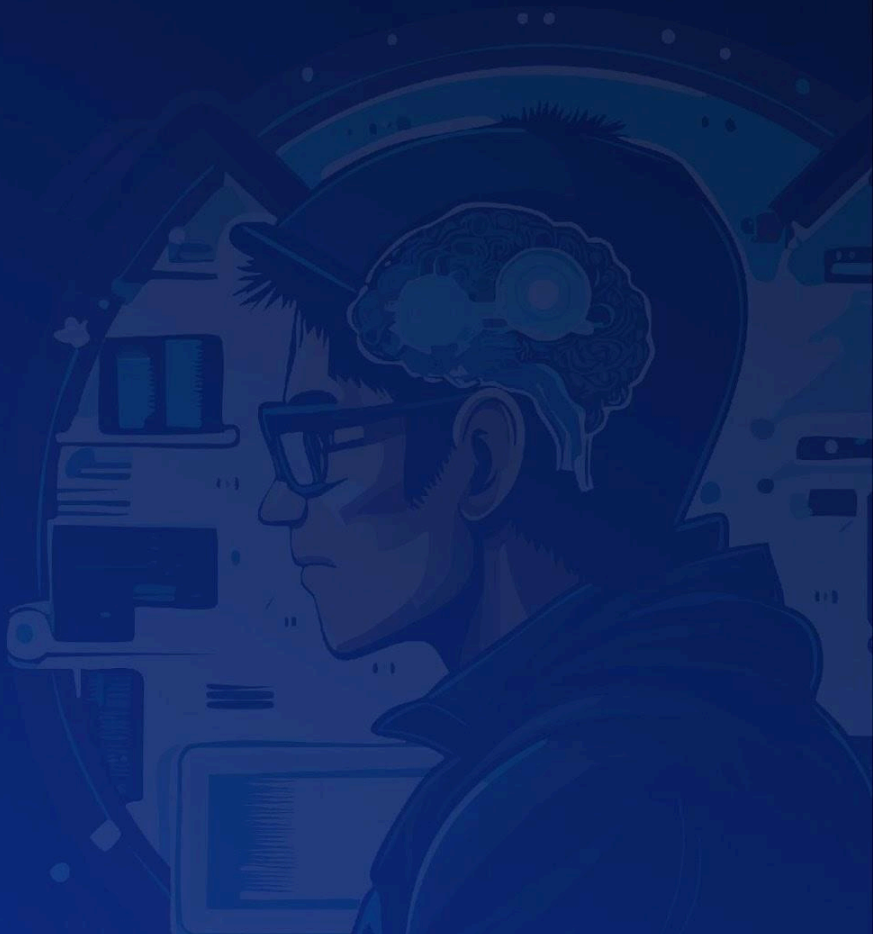


Table of Contents

Summary

Security Review

Felix Morpho-BLP adapter



Summary

Three Sigma audited Felix Morpho-BLP Adapter in a 1.2 person week engagement. The audit was conducted from 21/01/2026 to 23/01/2026.

Protocol Description

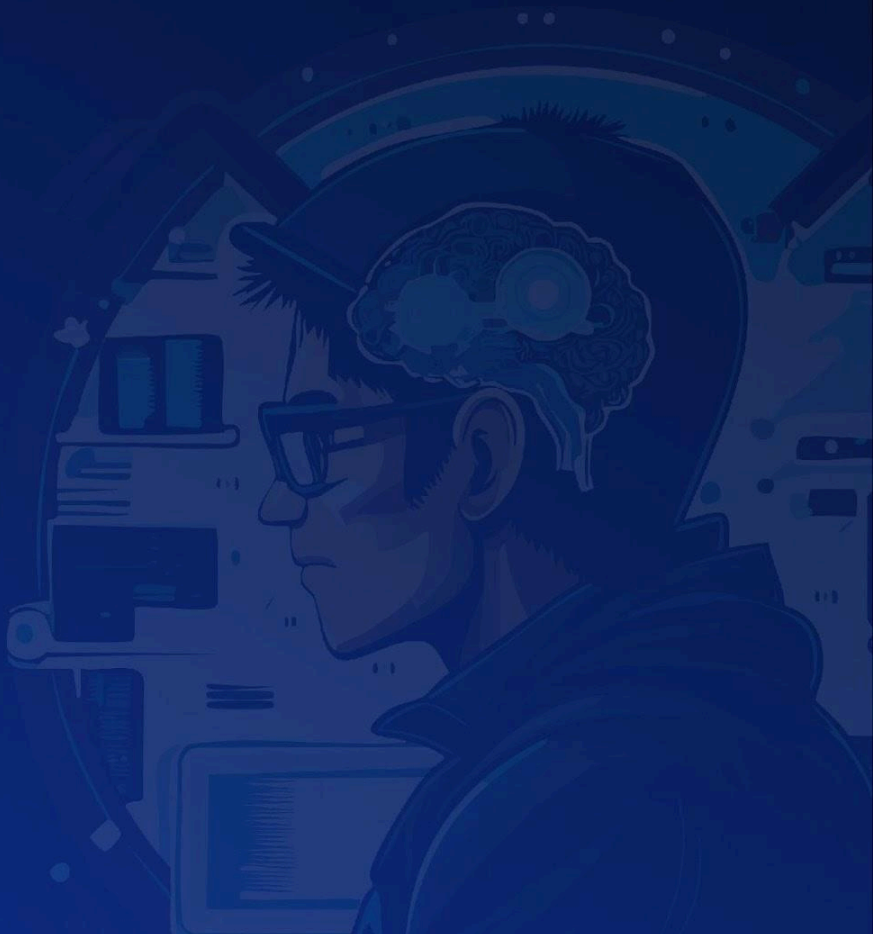
The Morpho-BLP Adapter integrates Hyperliquid's HyperCore Backstop Liquidity Provider (BLP) vaults with Morpho VaultV2, bridging Morpho's synchronous ERC-4626 interface with HyperCore's asynchronous block execution model. The adapter employs a declarative reconciliation pattern to manage assets across three locations: an EVM buffer holding liquid assets for immediate synchronous withdrawals, a transient HyperCore spot balance used during bridge operations, and the BLP vault itself where assets are invested to earn yield.

To handle the inherent mismatch between Morpho's synchronous withdrawal requirements and HyperCore's asynchronous settlement, the adapter maintains a configurable target buffer on the EVM side, ensuring that deallocate() calls can be served immediately. State transitions are advanced through a permissionless advance() function, rate-limited by a minimum interval, which reconciles the adapter's actual state toward its desired state in a self-healing manner. The adapter also uses snapshot-based accounting to prevent NAV dips caused by HyperCore's observation lag.

Scope

Security Review

Felix Morpho-BLP adapter



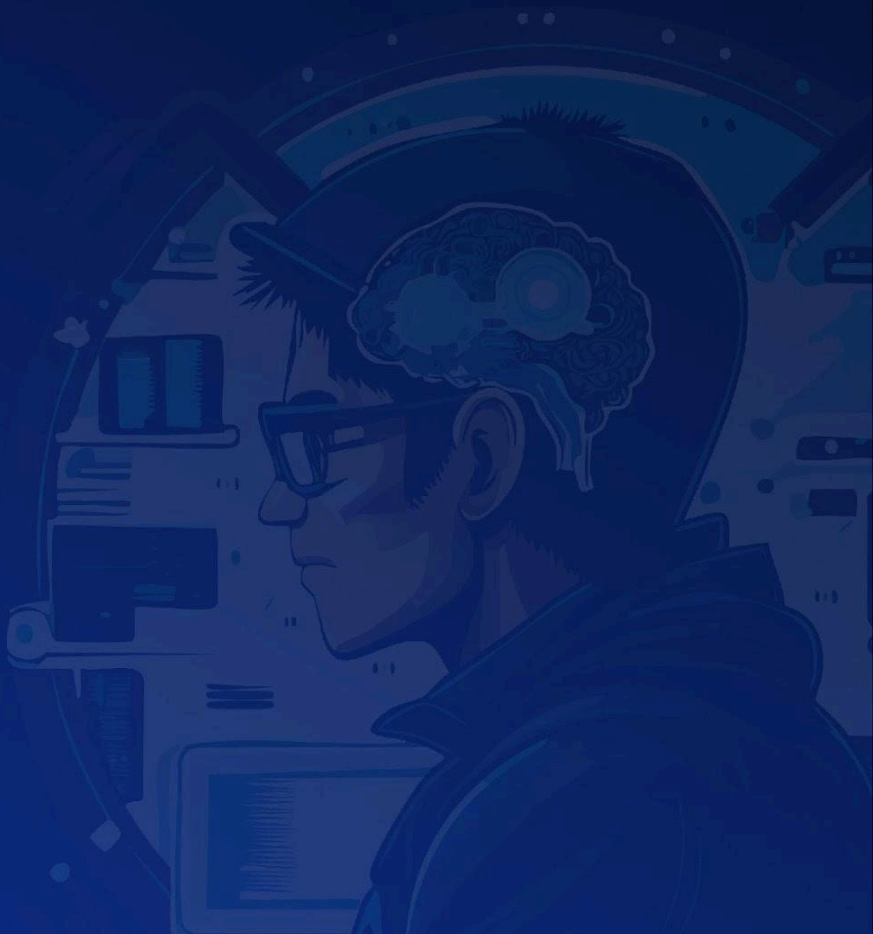
Scope

Filepath	nSLOC
src/HyperCoreBLPAdapter.sol	272
src/libraries/HyperCoreActionsLib.sol	229
src/libraries/ErrorsLib.sol	42
src/libraries/SafeERC20Lib.sol	23
Total	**566**

Methodology

Security Review

Felix Morpho-BLP adapter



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

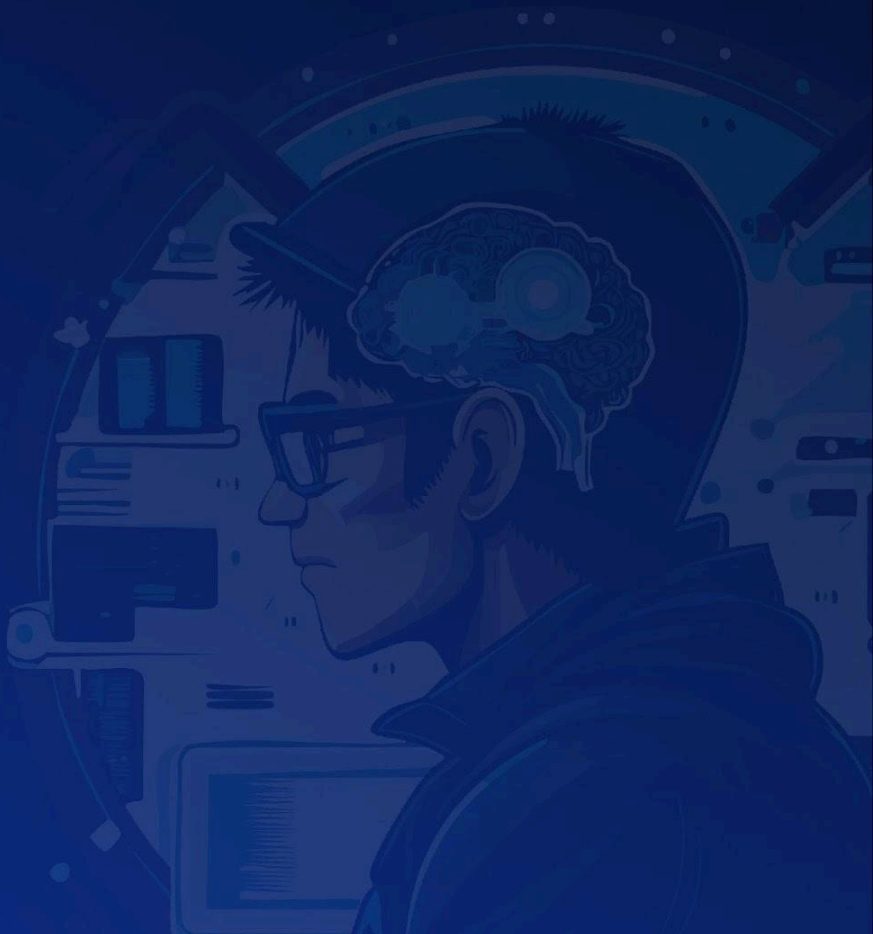
In this audit, we classify findings based on Immunefi's Vulnerability Severity Classification System (v2.3) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard

Security Review

Felix Morpho-BLP adapter



Project Dashboard

Application Summary

Name	Morpho Vault V2 HyperCore BLP Adapter
Repository	https://github.com/felixprotocol/morpho-vault-v2-blp-adapter
Commit	2c5a02e
Language	Solidity
Platform	HyperEVM

Engagement Summary

Timeline	21/01/2026 to 23/01/2026
№ of Auditors	2
Review Time	1.2 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	1	1	0
Medium	0	0	0
Low	0	0	0
None	5	5	0

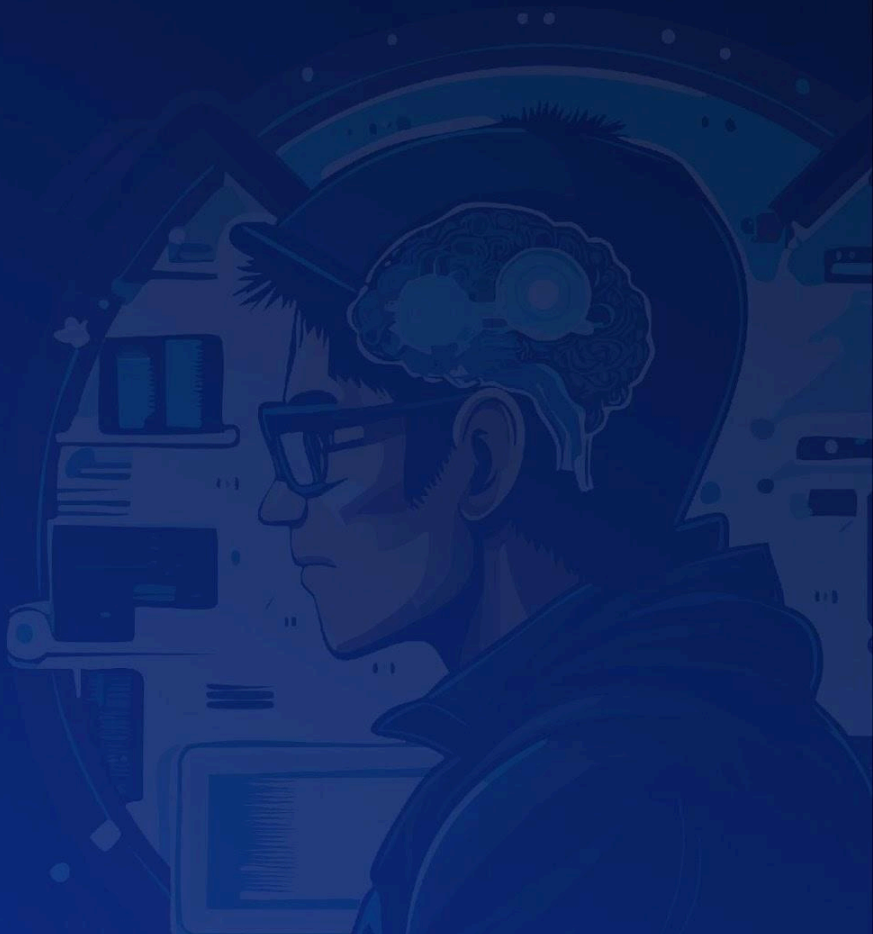
Category Breakdown

Suggestion	4
Documentation	1
Bug	1
Optimization	0
Good Code Practices	0

Risk Section

Security Review

Felix Morpho-BLP adapter



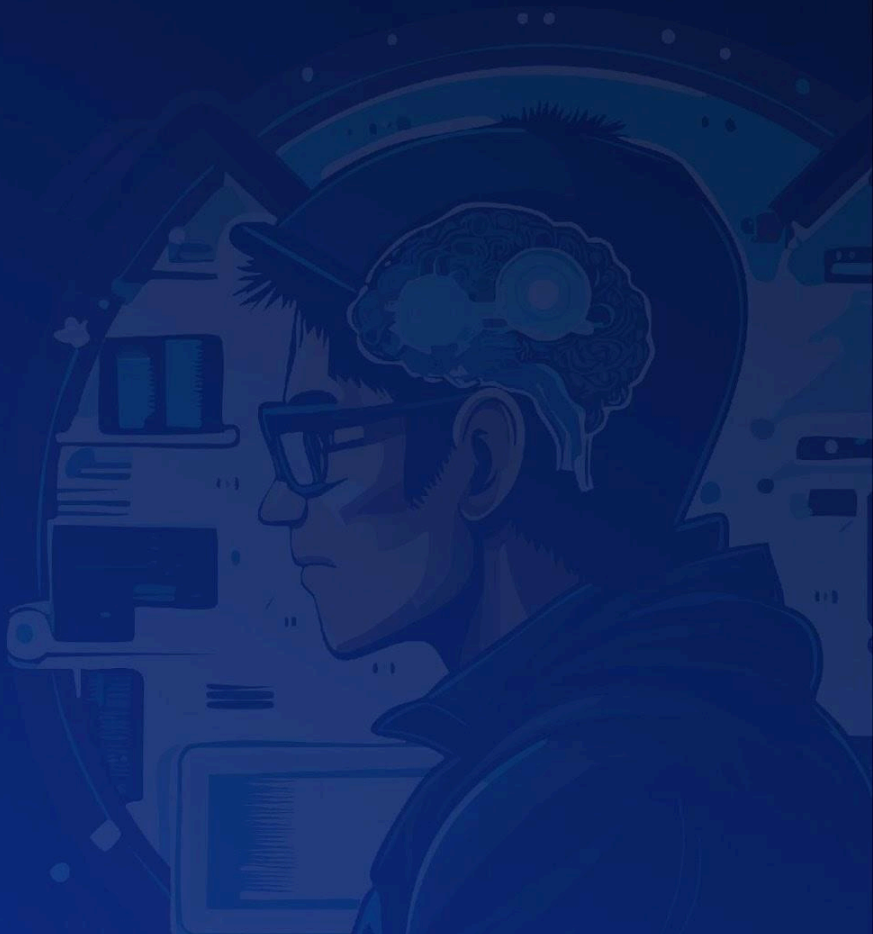
Risk Section

No risks were identified.

Findings

Security Review

Felix Morpho-BLP adapter



Findings

3S-Felix-H01

Flawed priority ordering in advance function enables denial of service on buffer replenishment via spot balance donations

Id	3S-Felix-H01
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #9bb15ce.

Description

The **HyperCoreBLPAdapter::advance** function implements a declarative reconciliation mechanism that moves the adapter toward its desired state through priority-ordered actions. The function manages three balance locations: EVM buffer, HyperCore spot, and BLP vault equity. The intended replenishment flow when the EVM buffer falls below **targetBufferAmount** is a two-step process: first withdraw from BLP to spot (Priority 3), then bridge from spot to EVM (Priority 1).

The current priority ordering is: (1) bridge spot to EVM if buffer is low, (2) deposit spot to BLP, (3) withdraw from BLP to spot if buffer is low, (4) invest excess EVM balance. This ordering assumes **coreSpot** equals zero when replenishment is needed, but this assumption can be violated by an external actor.

An attacker can exploit this by donating a minimal amount (e.g., 1 wei) to the adapter's HyperCore spot balance a few blocks before **advance** is called. Since HyperCore's core writer requires approximately two blocks to update the spot balance, the donation becomes observable when **advance** executes. The function then enters Priority 1, bridging only the attacker's insignificant donation to EVM instead of progressing through the legitimate replenishment flow. Because **advance** is rate-limited by **MIN_ADVANCE_INTERVAL**, the attacker can repeat this donation pattern indefinitely, preventing the adapter from ever completing buffer replenishment from BLP equity. This leaves the EVM buffer depleted, causing **deallocate** calls to revert with **AdapterInsufficientBuffer**, effectively blocking user withdrawals from the parent Morpho VaultV2.

Recommendation

Reorder the priorities in **HyperCoreBLPAdapter::advance** to prevent spot balance donations from disrupting the replenishment flow. The recommended priority structure is:

Priority 0: If **evmBalance + coreSpot + blpEquity < targetBufferAmount**, drain all positions to EVM to maximize available buffer.

Priority 1: If **evmBalance < targetBufferAmount** and **coreSpot >= targetBufferAmount - evmBalance**, bridge the required amount from spot to EVM.

Priority 2: If **evmBalance < targetBufferAmount** and **blpEquity >= targetBufferAmount - evmBalance - coreSpot**, withdraw the needed amount from BLP to spot.

Priority 3: If **coreSpot > 0** and **evmBalance > targetBufferAmount**, deposit spot funds to BLP.

Priority 4: If **evmBalance > targetBufferAmount + investThreshold**, invest excess to BLP.

This reordering ensures that when **evmBalance < targetBufferAmount**, the system initiates withdrawal from BLP (Priority 2) before depositing any spot balance to BLP (Priority 3). Small spot donations will not trigger Priority 1 unless **coreSpot >= targetBufferAmount - evmBalance**, and will instead be swept into BLP via Priority 3 when the buffer is healthy.

3S-Felix-N01

Misleading NatSpec comment in constructor incorrectly describes infinite approval behavior

Id	3S-Felix-N01
Classification	None
Category	Suggestion
Status	Addressed in #bfc196d.

Description

The **HyperCoreBLPAdapter** contract implements a two-step initialization pattern using a constructor for immutable state and an **initialize** function for upgradeable state setup. The constructor's NatSpec documentation contains an inaccurate **@dev** comment that describes functionality which actually occurs in a different function.

The constructor's **@dev** tag states: "Grants infinite approval to parentVault to enable synchronous deallocate via transferFrom". However, the constructor only sets immutable variables (**ASSET**, **HYPERCORE_VAULT**, **PARENT_VAULT**, **MIN_ADVANCE_INTERVAL**, **ADAPTER_ID**) and emits the **Genesis** event. The infinite approval to **PARENT_VAULT** is actually granted in the **HyperCoreBLPAdapter::initialize** function via:

SafeERC20Lib.safeApprove(ASSET, PARENT_VAULT, type(uint256).max);

This documentation discrepancy can lead to confusion for developers, auditors, and integrators who rely on NatSpec comments to understand contract behavior without reading the full implementation.

Recommendation

Move the **@dev** comment to the **initialize** function where the approval actually occurs.

```
    /// @param _asset The ERC20 token to manage (e.g., USDC address)
    /// @param _hypercoreVault The HyperCore BLP vault address for depositing assets
    /// @param _parentVault The Morpho VaultV2 address that will call this adapter
    /// @param _minAdvanceInterval Minimum seconds between advance() calls
(recommended: 60-300)
-   /// @dev Grants infinite approval to parentVault to enable synchronous deallocate via
```

```
transferFrom
    constructor(address _asset, address _hypercoreVault, address _parentVault, uint256
_minAdvanceInterval) {



+   /// @dev Grants infinite approval to parentVault to enable synchronous deallocate via
transferFrom
    function initialize(
        address _defaultAdmin,
        address _bufferSetter,
        address _investThresholdSetter,
        uint256 _targetBufferAmount,
        uint256 _investThreshold
    )
        external
        initializer
    {
```

3S-Felix-N02

Incomplete state documentation in **HyperCoreBLPAdapter::advance** priority 5 comment

Id	3S-Felix-N02
Classification	None
Category	Suggestion
Status	Addressed in #c72d27c.

Description

The **HyperCoreBLPAdapter::advance** function implements a declarative reconciliation mechanism that moves the adapter toward a desired state through a priority-ordered decision tree. The function evaluates the current state of assets across three locations (EVM buffer, HyperCore spot, and BLP vault) and executes one reconciliation action per call based on priority order.

The code comment for Priority 5 ("No Action Needed") documents only one scenario where the system reaches this state, but there is an additional valid scenario that is not documented. This creates a discrepancy between the code's actual behavior and its documentation.

The comment currently states that Priority 5 is reached when:

- **evmBalance** is in the range **[targetBufferAmount, targetBufferAmount + investThreshold]** AND **coreSpot == 0**

However, the function can also reach Priority 5 when:

- **evmBalance** is in the range **[0, targetBufferAmount)** AND **coreSpot == 0** AND **blpEquity == 0**

This second scenario occurs when the adapter has insufficient EVM balance to meet the target buffer, no spot funds on HyperCore, and no equity in the BLP vault. In this case, no reconciliation action can be taken because there are no funds available to replenish the buffer.

Recommendation

Update the Priority 5 comment to accurately document all scenarios that lead to this state.

```
//
```

```
        // PRIORITY 5: No Action Needed (Desired State Achieved)
        //
        // Context: Buffer is within acceptable range, no idle spot funds
-       // State: evmBalance ∈ [targetBufferAmount, targetBufferAmount + investThreshold]
-       //        AND coreSpot == 0
+       // State: One of the following:
+       //        1. evmBalance ∈ [targetBufferAmount, targetBufferAmount + investThreshold]
AND coreSpot == 0
+       //        2. evmBalance ∈ [0, targetBufferAmount) AND coreSpot == 0 AND blpEquity
== 0
        // Interpretation: System is in desired state, no reconciliation needed
```

3S-Felix-N03

Adapter deviates from Morpho Vault V2 specification by entering/exiting markets outside of **allocate**/**deallocate**

Id	3S-Felix-N03
Classification	None
Category	Suggestion
Status	Addressed in #a2a0f2a.

Description

The **HyperCoreBLPAdapter** contract integrates Hyperliquid's HyperCore BLP vault with Morpho VaultV2 using a state-driven approach. Due to the asynchronous nature of HyperCore operations, the adapter uses a permissionless **advance()** function to progressively move assets between the EVM buffer, HyperCore spot balance, and the BLP vault.

According to the Morpho Vault V2 specification, adapters must adhere to the following requirement:

> "They must enter/exit markets only in allocate/deallocate."

However, **HyperCoreBLPAdapter::advance()** performs market entry and exit operations (bridging assets to/from HyperCore and depositing/withdrawing from the BLP vault) outside of the **allocate()** and **deallocate()** functions. This is a necessary design choice to accommodate the async settlement model of HyperCore, where operations cannot complete synchronously within a single transaction.

This deviation does not introduce a security vulnerability, as the adapter's reconciliation logic is designed to be safe when called repeatedly and capable of automatically recovering toward the desired state. However, it represents a documented departure from the Morpho Vault V2 adapter specification.

Recommendation

Add a notice in the contract's NatSpec documentation explicitly acknowledging this deviation from the Morpho Vault V2 specification. For example, update the contract-level documentation to include:

/// @dev SPECIFICATION DEVIATION: This adapter enters/exits markets (BLP vault) in advance()

```
///     rather than exclusively in allocate()/deallocate(). This is required due to HyperCore's
///     asynchronous settlement model where operations cannot complete within a single
transaction.
///     See:
https://github.com/morpho-org/vault-v2/blob/5fecc5b83a0cb12997007416764360c4e367f27
3/src/VaultV2.sol#L69
```

3S-Felix-N04

Missing **_disableInitializers** call in constructor allows implementation contract initialization

Id	3S-Felix-N04
Classification	None
Category	Suggestion
Status	Addressed in #e066347.

Description

The **HyperCoreBLPAdapter** contract inherits from **AccessControlUpgradeable** and follows the upgradeable proxy pattern, separating deployment into a constructor for immutable variables and an **initialize** function for mutable state setup. The **initialize** function uses the **initializer** modifier to ensure it can only be called once per proxy instance.

However, the constructor does not call **_disableInitializers()**, which is the recommended practice for upgradeable contracts according to OpenZeppelin's guidelines. This omission allows an attacker to call **initialize** directly on the implementation contract (not the proxy). By doing so, the attacker can grant themselves **DEFAULT_ADMIN_ROLE**, **BUFFER_SETTER_ROLE**, and **INVEST_THRESHOLDER_ROLE** on the implementation contract. While this does not directly affect proxies pointing to this implementation, it violates the security principle that implementation contracts should never be left in an initialized state. In certain scenarios involving selfdestruct mechanics or implementation contract interactions, an initialized implementation could pose unexpected risks.

Recommendation

Call **_disableInitializers()** in the constructor to prevent the implementation contract from being initialized.

```
  constructor(address _asset, address _hypercoreVault, address _parentVault, uint256
_minAdvanceInterval) {
+     _disableInitializers();
      if (_asset == address(0)) revert ZeroAddress();
      if (_hypercoreVault == address(0)) revert ZeroAddress();
      if (_parentVault == address(0)) revert ZeroAddress();
      if (_minAdvanceInterval == 0) revert ZeroAmount();
      ASSET = _asset;
```

```
        HYPERCORE_VAULT = _hypercoreVault;
        PARENT_VAULT = _parentVault;
        MIN_ADVANCE_INTERVAL = _minAdvanceInterval;
        ADAPTER_ID = keccak256(abi.encode(_hypercoreVault));
        emit Genesis(ASSET, HYPERCORE_VAULT, PARENT_VAULT,
MIN_ADVANCE_INTERVAL);
    }
```

3S-Felix-N05

Incorrect acronym expansion in documentation leads to developer confusion

Id	3S-Felix-N05
Classification	None
Category	Documentation
Status	Addressed in #35156f9.

Description

The **README.md** file contains an incorrect expansion of the BLP acronym. On line 11, the documentation states that BLP stands for "Backstop Liquidity Provider":

This adapter enables Morpho VaultV2 to deploy capital into Hyperliquid's HyperCore BLP (Backstop Liquidity Provider)

However, the contract code in **HyperCoreBLPAdapter::getBLPEquity** interacts with **HyperCoreActionsLib.borrowLendUserState**, and the underlying HyperCore library defines **BorrowLendUserTokenState** as the data structure for BLP positions. This indicates that BLP refers to "Borrow Lending Protocol", not "Backstop Liquidity Provider". This terminology mismatch between documentation and the actual protocol can mislead developers during integration and auditing.

Recommendation

Update the README to use the correct acronym expansion.

- This adapter enables Morpho VaultV2 to deploy capital into Hyperliquid's HyperCore BLP (Backstop Liquidity Provider)
+ This adapter enables Morpho VaultV2 to deploy capital into Hyperliquid's HyperCore BLP (Borrow Lending Protocol)